

CC3001 Control 2 Primavera 2025

Profs. Valentina Aravena, Nelson Baloian y Patricio Poblete

21 de noviembre de 2025

Instrucciones:

- Tiempo: 2 horas
- El control es INDIVIDUAL
- Entregue cada pregunta en una hoja separada
- No olvide poner su nombre en cada hoja
- Está permitido usar apuntes, ya sea en papel o en formato digital
- Si usa un dispositivo para revisar sus apuntes en formato digital, está completamente prohibido utilizarlo para cualquier otra finalidad. *En particular, no se puede usar chat, ni mail ni ninguna otra forma de comunicación con otra persona o inteligencia artificial.* Tampoco puede usarlo para fotografiar el enunciado.

P1. Nombre: _____

1. [2 puntos] Comenzando con un árbol AVL vacío, inserte la secuencia de llaves 6, 5, 2, 1, 4, 3, indicando cada una de las rotaciones que necesite hacer:

2. [2 puntos] Repita lo anterior, usando un árbol 2-3

3. [2 puntos] Considere una tabla de hashing con Linear Probing con $m = 10$ y la función de hashing $h(x) = x \% 10$ y muestre la tabla resultante al insertar la siguiente secuencia de llaves:

40, 23, 67, 48, 37, 58, 10, 87

P2. Nombre: _____

Suponga que se desea ordenar con Quicksort un arreglo que puede tener valores duplicados. Para esto, vamos a suponer que existe una función *particion*(a, i, j) que particiona el subarreglo $a[i], \dots, a[j]$. Para esto, elige un pivote al azar, y luego particiona dejando a la izquierda los elementos menores que el pivote, al centro los elementos iguales al pivote y a la derecha los mayores que el pivote. La función retorna una tupla $(k1, k2)$ tal que los elementos iguales al pivote están en los casilleros $a[k1], \dots, a[k2]$.

1. Modifique el algoritmo *qsort* del apunte (y que aparece a continuación) para hacer uso de esta nueva función *particion*. Escriba su algoritmo modificado en el espacio a la derecha.

```
def quicksort(a):  
    qsort(a, 0, len(a)-1)  
  
# ordena a[i], ..., a[j]  
def qsort(a, i, j):  
    if i < j:  
        k = particion(a, i, j)  
        qsort(a, i, k-1)  
        qsort(a, k+1, j)
```

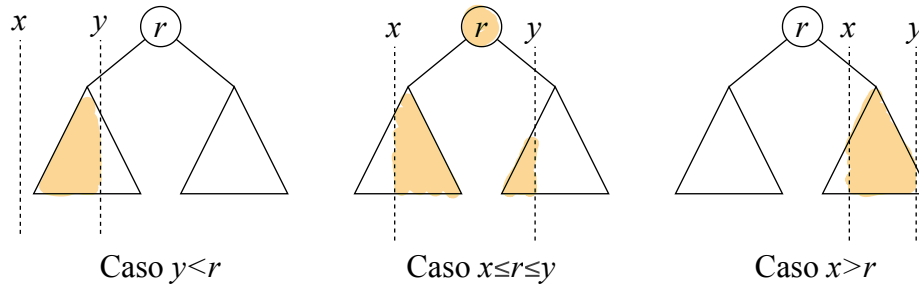
2. Implemente este nuevo algoritmo *particion* utilizando el siguiente invariante:

$< p$	$= p$	$> p$	$?$
i	$k1$	$k2$	d
			j

P3. Nombre: _____

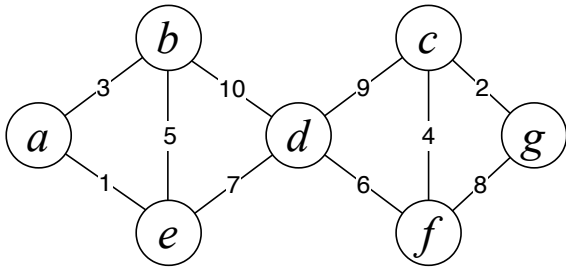
Se desea “podar” un árbol de búsqueda binaria cuya raíz es apuntada por un puntero p , dejando solo sus nodos que están en un rango dado $[x..y]$. Esto debe hacerse en tiempo proporcional a la altura del árbol. Escriba una función $podar(p, x, y)$ que transforme el árbol original en el árbol podado y que retorne un puntero a la raíz del árbol resultante. Su algoritmo no debe crear nuevos nodos, solo debe reenlazar los nodos del árbol original.

Sugerencia: Considere los tres casos siguientes (aparte del caso base):

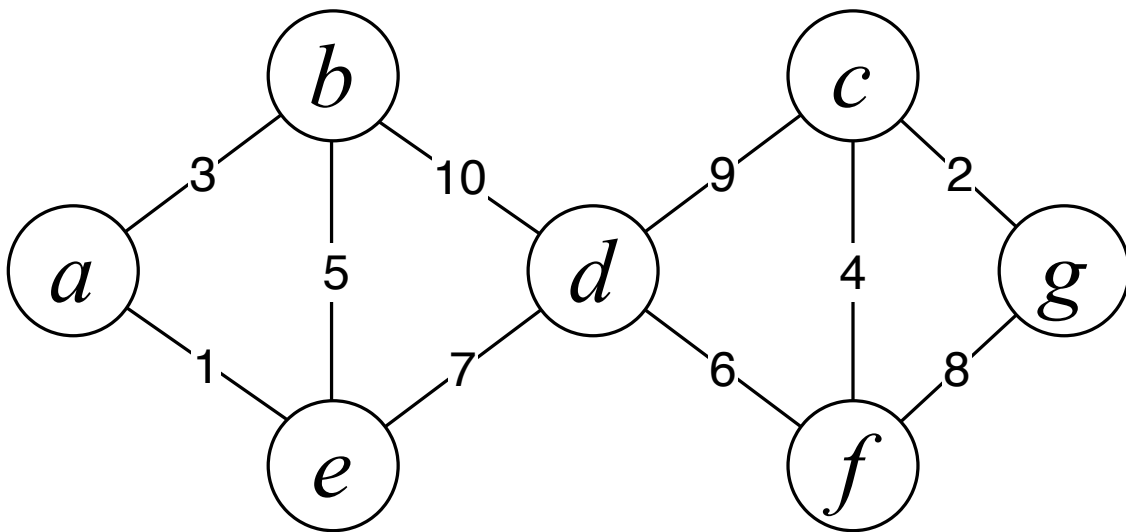


P4. Nombre: _____

Considere el siguiente grafo, en que los números indican los costos de los arcos respectivos:



1. Aplique el algoritmo de Kruskal para encontrar un árbol cobertor de costo mínimo. Destaque con trazo grueso los arcos del árbol resultante y agregue una numeración al lado de sus arcos para indicar el orden en que se fueron agregando al árbol.



2. Repita lo anterior usando el algoritmo de Prim.

